Reinforcement Learning

Introduction

Reinforcement learning is a framework for solving control tasks (also called decision problems) by building agents that learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.

The RL loop outputs a sequence of **state, action, reward and next state.**

- Our Agent receives **state** s_0 from the **Environment**
- the Agent decide which **action** to take following a policy $\pi(a|s_0)$
- The agent goes to a **new state** s_1
- The environment gives some **reward** r_1 to the Agent

The objective of Reinforcement Learning is to **learn an optimal policy** π^* that maximizes the expected return: $\pi^* = \arg\max_{\pi} \mathbb{E}_{\tau\sim\pi}\big[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}\big]$

γ is a discount rate that reduce the value of long-term rewards:

- if γ is large (es. 0.99) our **agent cares more about the long-term reward.**
- if γ is small (es. 0.95) our **agent cares more about the short term reward**

PPO

The Proximal Policy Optimization is a new family of policy gradient methods for reinforcement learning. Policy gradient methods often leads to destructively large policy update. PPO uses clipped probability ratios, which forms a pessimistic estimate (i.e. lower bound) of the performance of the policy.
PPO alternate between sampling data through interaction with the environment, and optimizing a “surrogate” objective function using stochastic gradient ascent. It implements some improvement to the trust region policy optimization (TRPO) whose surrogate objective function is:

$$\mathcal{L}^{CPI}(\theta) = \mathbb{E}_t[r_t(\theta)\hat{A}_t]$$

- Probability ratio: it measures how much the new policy changes the probability of the action that was actually taken, compared to the old policy

that generated the data.
$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$

- Advantage estimator: can be defined as the difference between the comulative reward and the value function in the state s_t. It quantify how much the action was good or bad
$\hat{A}_t = G_t - V(s_t) \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_k$

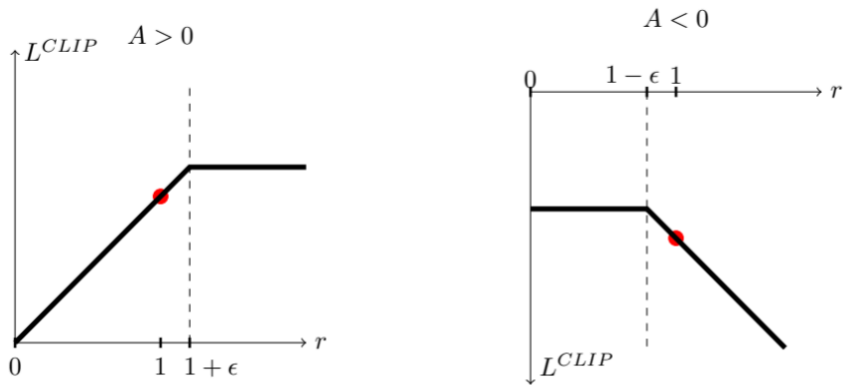
Clipped Surrogate Objective

Maximization of $\mathcal{L}^{CPI}$ can lead to an excessively large policy update, instead with clipping we obtain:

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

It penalize changes to the policy that move $r_t(\theta)$ away from 1
→ clip $r_t(\theta)$ in the interval $[1-\epsilon, 1+\epsilon]$

Take the minimum of the clipped and unclipped objective, the final objective is a lower bound of the unclipped objective.



Adaptive KL Penalty Coefficient

Another approach, which can be used as an alternative to the clipped surrogate objective, or in addition to it, is to use a penalty on KL divergence, and to adapt the penalty coefficient β.

$$\mathcal{L}^{KLPEN}(\theta) = \mathbb{E}_t\Big[r_t(\theta)\hat{A}_t - \beta KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]\Big]$$

- $d = \mathbb{E}_t\Big[KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]\Big]$ measure the change between the new and the old policy at the state s_t
- β is the weight of the KL penalization, its value is not fixed but is adapted to maintain KL near to a target value d_{targ}
  - if $d < d_{targ}/1.5$, $\beta \leftarrow \beta/2$
  - if $d > d_{targ} \times 1.5$, $\beta \leftarrow \beta \times 2$

  We occasionally see policy updates where the KL divergence is significantly different from d_{targ} , however, these are rare, and β quickly adjusts.

  The initial value of β is a another hyperparameter but is not important in practice because the algorithm quickly adjusts it.

Algorithm

For the implementation we can use a loss function that combines the policy surrogate ($\mathcal{L}^{CLIP}$ or $\mathcal{L}^{KLPEN}$) and a value function error term. This objective can further be augmented by adding an entropy bonus to ensure sufficient exploration.

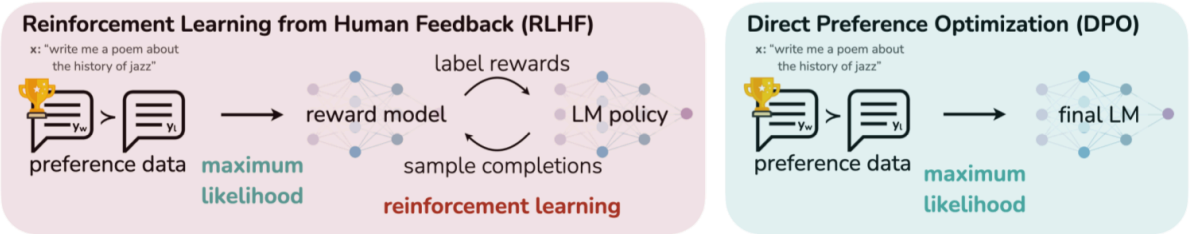
$$\mathcal{L}_t^{CLIP+VF+S}(\theta) = \mathbb{E}_t[\mathcal{L}_t^{CLIP}(\theta) - c_1\mathcal{L}_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]$$

- c_1, c_2 are coefficients, S denotes an entropy bonus
- $\mathcal{L}_t^{VF} = (V_\theta(s_t) - V_t^{targ})^2$ is a squared-error loss

> for iteration=1, 2, . . . do
>
> for actor=1, 2, . . . , N do
>
> Run policy $\pi_{\theta_{old}}$ in environment for T timesteps
>
> Compute advantage estimates $\hat{A}_1, ..., \hat{A}_T$
>
> end for
>
> Optimize surrogate $\mathcal{L}$ wrt θ, with K epochs and minibatch size M ≤ NT
>
> $\theta_{old} \leftarrow \theta$
>
> end for

DPO

- DPO is a new parameterization of the reward model in RLHF that use only a simple classification loss.
- It directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an implicit reward model whose corresponding optimal policy can be extracted in closed form.
- It uses a change of variables to define the preference loss as a function of the policy directly. In essence, the policy network represents both the language model and the (implicit) reward.
- In this way we don't have to learn a reward model and then optimize it, like prior RLHF methods.



DPO Loss function

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

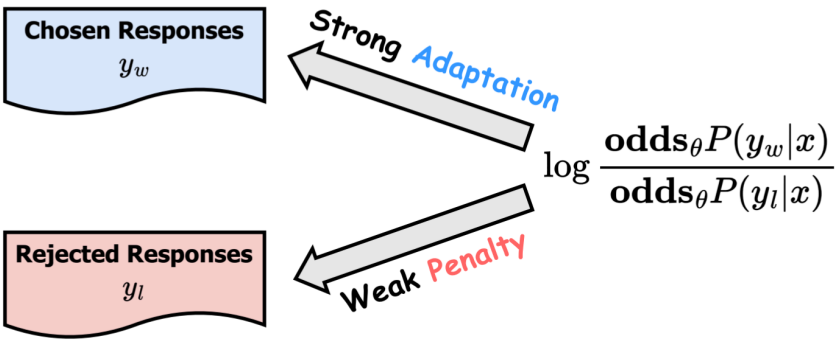
- π_{ref}: base reference policy i.e. the initial SFT model π^{SFT} (Supervised fine-tuning)
- $\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}$ how much π_θ favors y_w respect to π_{ref}
- $\beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$ how much π_θ favors y_l respect to π_{ref}
- β is a parameters controlling the derivation from π_{ref}
- σ: sigmoid function, probability that the model prefers y_w respect to y_l
- $-\log\sigma(\cdot)$: binary cross-entropy loss of how much the π_θ favor y_w respect to y_l, more than what π_{ref} was doing.

DPO directly trains the policy so that it assigns higher probability to responses preferred by humans compared to those that are rejected, without using a reward model, while remaining close to a reference model.

ORPO

ORPO aligns the language model without a reference model in a single-step manner by assigning a weak penalty to the rejected responses and a strong adaptation signal to the chosen responses with a simple log odds ratio term appended to the negative log-likelihood loss.

- Use only a minor penalty for the disfavored generation style
- No additional preferences alignment phase is required, SFT warm-up
- No references model is required



Odds Ratio Preference Optimization

ORPO Loss function

$$\mathcal{L}_{ORPO} = \mathbb{E}_{(x,y_w,y_l)}[\mathcal{L}_{SFT} + \lambda \cdot \mathcal{L}_{OR}]$$

- $\mathcal{L}_{SFT}$ supervised fine-tuning loss
- $\mathcal{L}_{OR} = -\log \sigma\left(\log \frac{odds_\theta(y_w|x)}{odds_\theta(y_l|x)}\right)$

  maximizes the odds ratio between the likelihood of generating the favored response y_w and the disfavored response y_l

  - $odds_\theta(y|x) = \frac{P_\theta(y|x)}{1-P_\theta(y|x)}$
- Together, $\mathcal{L}_{SFT}$ and $\mathcal{L}_{OR}$ weighted with λ, tailor the pre-trained language model to adapt to the specific subset of the desired domain and disfavor generations in the rejected response sets.

RLOO

Reinforcement Leave-One-Out (RLOO) revisit the formulation of alignment from human preferences in the context of RL starting from PPO.

PPO was designed for a regime where off-policy gradient updates are large enough to introduce instability, but in the setting of RLHF when we fine-tune a pre-trained LLM we have a lack of this characteristic.

- PPO is not the right tool for doing RL in RLHF: the “basic” vanilla Policy Gradient REINFORCEMENT is better
- RLOO outperforms key baselines: it enables using multiple online samples
- Modeling partial completions is not necessary: modeling the full generations preserves performance while reducing complexity in the RL stage and significantly accelerating learning.
- RLOO is relatively robust to noise and KL penalty sensitivity

Reinforce Estimator

Given that in LLM applications, the reward value is only obtained at the end of the full sequence we can model the entire generation as a single action → use the REINFORCE estimator.

$$\mathbb{E}_{x \sim D, y \sim \pi_\theta(.|x)}[R(y,x) \nabla_\theta \log \pi_\theta(y|x)]$$

REINFORCE estimator allow for:

- back-propagation through the discrete action space
- directly optimize the KL-shaped reward objective $R(x,y) = r_\phi(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$
  for the entire sequence

To improve learning, one can reduce the variance of the estimator, while keeping it unbiased, by subtracting a baseline b that has high covariance with the stochastic gradient estimate.

$$\mathbb{E}_{x \sim D, y \sim \pi_\theta(.|x)}[(R(y,x) - b) \nabla_\theta \log \pi_\theta(y|x)]$$

We can use as baseline the moving average of all rewards throughout training: $b_{MA} = \frac{1}{S} \sum_s R(x^s, y^s)$ where S is the number of training steps, and (x^s, y^s) is the prompt-completion pair at the step s.

Reinforcement Leave-One-Out

If we have access to multiple online samples, the previous baseline b can be improved:

- The reward for each sample can serve all other samples as a baseline
- Policy updates can be done on an average of gradient estimates for each sample

The baseline for sample i is the mean of all others sample reward

$b_i = \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x)$

RLOO estimator:

$$\frac{1}{k} \sum_{i=1}^{k} \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x) \right] \nabla \log \pi(y_{(i)} | x)$$

for $y_{(1)}, ..., y_{(k)} \sim \pi_\theta(.|x)$

Where k refers to the number of online samples generated

This is a much more effective baseline than b_{MA} since it's created on-the-fly for each sample and at each training step, but comes at a cost of increased sampling time during training.